

08 - 技能与工具

阅读时间：约 2 小时
 前置知识：07 - 记忆系统实战
 学习目标：掌握 Nanobot 的 Skill 系统、内置工具体系、MCP 工具集成，能够自定义 Skill

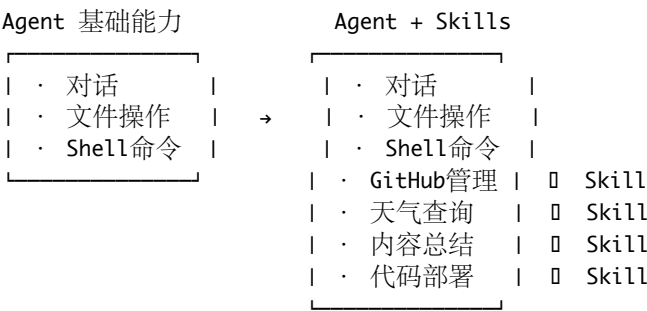
目录

- 8.1 Skills 系统概述
- 8.2 SKILL.md 格式规范
- 8.3 YAML Frontmatter 字段详解
- 8.4 技能发现机制
- 8.5 渐进披露 (Progressive Disclosure)
- 8.6 内置技能列表
- 8.7 内置工具完整列表
- 8.8 ToolRegistry 统一注册与执行机制
- 8.9 MCP 工具集成
- 8.10 自定义 Skill 编写实战
- 8.11 工具安全机制
- 8.12 面试高频题
- 8.13 本章小结

8.1 Skills 系统概述

8.1.1 什么是 Skill?

在 Nanobot 中，Skill（技能）是一种可插拔的能力扩展机制。你可以把它理解为”给 Agent 添加的专业模块”：



8.1.2 Skill vs Tool 的区别

这是面试中容易混淆的概念：

维度	Skill（技能）	Tool（工具）
粒度	粗粒度（一个完整能力）	细粒度（一个具体操作）
定义方式	Markdown 文件（SKILL.md）	Python 代码注册
包含内容	指令 + 脚本 + 参考资料	函数签名 + 执行逻辑
注入方式	System Prompt	Tool Definition
类比	一门专业课程	一个具体工具

关系：一个 Skill 可以教会 Agent 如何更好地使用多个 Tool。

Skill: GitHub 管理

└─ 知道如何创建 PR（指令）

- └─ 知道代码审查流程（知识）
 - └─ 会使用以下 Tools:
 - └─ exec（运行 git 命令）
 - └─ read_file（读取代码）
 - └─ web_fetch（查看 GitHub API）
-

8.2 SKILL.md 格式规范

8.2.1 目录结构

每个 Skill 是一个独立的目录：

```
skill-name/  
└─ SKILL.md          # 技能定义文件（必需）  
└─ scripts/         # 可选：辅助脚本  
    └─ deploy.sh  
    └─ check.py  
└─ references/      # 可选：参考资料  
    └─ api-docs.md  
    └─ examples.md  
└─ assets/          # 可选：资源文件  
    └─ template.json  
    └─ config.yaml
```

8.2.2 SKILL.md 文件格式

SKILL.md 由两部分组成：**YAML Frontmatter** + **Markdown** 正文。

```
---  
name: github  
description: "GitHub 仓库管理、PR 创建与代码审查"  
always: false  
metadata: '{"nanobot.requires.bins": ["git", "gh"], "nanobot.requires.env": ["GITHUB_TOKEN"]}'  
---
```

GitHub 管理技能

能力

你可以帮助用户管理 GitHub 仓库，包括：

1. **** 创建和管理 Pull Request ****
 - 创建 PR 并添加描述
 - 审查代码变更
 - 合并 PR
2. **** Issue 管理 ****
 - 创建 Issue
 - 标签分类
 - 分配负责人

使用步骤

创建 PR

1. 先用 `exec` 运行 `git status` 检查当前状态
2. 确认所有变更已提交

3. 使用 ``exec`` 运行 ``gh pr create --title " 标题" --body " 描述"``

代码审查

- 1. 用 ``exec`` 运行 ``gh pr diff <PR 号>``
- 2. 用 ``read_file`` 查看关键变更文件
- 3. 给出审查意见

注意事项

- 创建 PR 前确保分支已推送到远程
- 大型 PR 建议拆分为多个小 PR
- 代码审查时关注安全和性能问题

参考资料

更多细节请查看 ``references/`` 目录中的文档。

8.3 YAML Frontmatter 字段详解

8.3.1 必填字段

字段	类型	说明	示例
name	string	技能的唯一标识符	"github"
description	string	技能的简短描述（用于摘要展示）	"GitHub 仓库管理"

8.3.2 可选字段

字段	类型	默认值	说明
always	bool	false	是否每轮对话都全文注入 System Prompt
metadata	string (JSON)	null	技能的元数据，包括依赖检查

8.3.3 always 字段详解

`always: true` → 每轮对话都将 `SKILL.md` 全文注入 System Prompt

适用于: Agent 核心能力，必须时刻知道的技能

`always: true`

`always: false` (默认) → 仅注入摘要，需要时 Agent 主动 `read_file` 查看全文

适用于: 偶尔使用的技能，节省 token

`always: false`

设计权衡:

`always: true`

- └─ 优点: Agent 始终知道完整技能内容
- └─ 缺点: 消耗更多 token
- └─ 适用: 核心技能 (1-2个)

`always: false`

- └─ 优点: 节省 token (仅注入一行摘要)
- └─ 缺点: Agent 需要额外步骤读取全文
- └─ 适用: 大多数技能

8.3.4 metadata 字段详解

metadata 是一个 JSON 字符串，用于声明技能的依赖：

```
metadata: '{"nanobot.requires.bins": ["git", "gh"], "nanobot.requires.env": ["GITHUB_TOKEN"]}'
```

元数据 Key	说明	示例
nanobot.requires.bins	需要的系统命令	["git", "gh", "docker"]
nanobot.requires.env	需要的环境变量	["GITHUB_TOKEN", "AWS_KEY"]

依赖检查流程：

技能加载时：

1. 检查 nanobot.requires.bins
 - 遍历列表，对每个 bin 执行 `which <bin>`
 - 如果命令不存在，打印警告但不阻止加载
2. 检查 nanobot.requires.env
 - 遍历列表，检查 `os.environ.get(<env>)`
 - 如果环境变量未设置，打印警告

注意：依赖缺失不会阻止技能加载，只会打印警告。
技能可能在运行时因依赖缺失而失败，此时 Agent 会收到错误信息并处理。

8.4 技能发现机制

8.4.1 搜索路径

Nanobot 在两个位置搜索技能：

搜索顺序（优先级从高到低）：

1. `<workspace>/skills/` □ 用户自定义技能（优先）
2. `nanobot/skills/` □ 内置技能（包内预置）

同名技能：workspace 覆盖内置

8.4.2 发现流程

简化的技能发现逻辑

```
def discover_skills(workspace: str) -> dict:
    skills = {}

    # 1. 先加载内置技能
    builtin_skills_dir = os.path.join(NANOBOT_PACKAGE_DIR, "skills")
    for skill_dir in os.listdir(builtin_skills_dir):
        skill = load_skill(os.path.join(builtin_skills_dir, skill_dir))
        if skill:
            skills[skill.name] = skill

    # 2. 再加载 workspace 技能（同名覆盖内置）
    workspace_skills_dir = os.path.join(workspace, "skills")
    if os.path.exists(workspace_skills_dir):
        for skill_dir in os.listdir(workspace_skills_dir):
            skill = load_skill(os.path.join(workspace_skills_dir, skill_dir))
            if skill:
```

```
skills[skill.name] = skill # 覆盖同名内置技能
```

```
return skills
```

8.4.3 覆盖机制

这个设计允许用户定制内置技能：

内置 github 技能：

nanobot/skills/github/SKILL.md → "使用 gh 命令管理 GitHub"

用户自定义覆盖：

workspace/skills/github/SKILL.md → "使用 GitHub API 管理，需要审批流程"

最终生效：用户自定义版本

面试要点：这种“约定优于配置 + 同名覆盖”的设计模式，在很多框架中都能看到（如 Maven 的 Convention over Configuration，Webpack 的 resolve 策略）。

8.5 渐进披露 (Progressive Disclosure)

8.5.1 为什么需要渐进披露

如果有 20 个技能，每个 SKILL.md 有 500 行，全部注入 System Prompt 就是 10000 行——这会：

1. 消耗大量 token（费钱）
2. 分散 Agent 注意力（效果差）
3. 可能超出上下文窗口（直接报错）

8.5.2 三层渐进披露设计

Tier 1: 摘要目录（始终注入）

所有技能的 name + description 组成的摘要列表

消耗：约 100-500 tokens

示例注入内容：

"你拥有以下技能：

- github: GitHub 仓库管理、PR 创建与代码审查
- weather: 查询全球天气信息
- summarize: 长文本智能摘要
- tmux: 终端多窗口管理

如需使用某个技能，请先读取对应的 SKILL.md 了解详情。"

```
|
| 当 Agent 判断需要使用某个技能时
|
```

Tier 2: 读取 SKILL.md 全文（按需加载）

Agent 主动调用 read_file 读取完整 SKILL.md

消耗：按需，约 200-1000 tokens

Agent 内部思考：

"用户想管理 GitHub PR，我需要查看 github 技能的详细说明"

[调用工具: read_file]

[路径: skills/github/SKILL.md]

```
|
| 如果需要更详细的参考资料或脚本
|
```

Tier 3: 访问 `scripts/references` (深度使用)

Agent 读取辅助脚本或参考文档
消耗: 按需

[调用工具: `read_file`]
[路径: `skills/github/references/pr-workflow.md`]
[调用工具: `exec`]
[命令: `bash skills/github/scripts/create-pr.sh`]

8.5.3 `always: true` 的特殊处理

标记为 `always: true` 的技能跳过渐进披露，直接全文注入：

技能注入逻辑：

```
|
| 遍历所有技能
|
| — always: true → 全文注入 System Prompt
|
| — always: false → 仅注入 name + description 摘要
```

面试加分：渐进披露是 UI/UX 设计中的经典原则——先展示概要，让用户按需深入。Nanobot 将这个原则应用到了 Agent Prompt 设计中，是一种优雅的 token 管理策略。

8.6 内置技能列表

Nanobot 预置了以下技能：

技能名	功能	always	依赖
github	GitHub 仓库管理、PR、Issue	false	git, gh, GITHUB_TOKEN
weather	全球天气查询	false	无
summarize	长文本智能摘要	false	无
tmux	终端多窗口管理	false	tmux
clawhub	ClawHub 平台集成	false	无
skill-creator	帮助用户创建新技能	false	无

各技能详解

github 技能：

能力：

- 克隆/创建/管理仓库
- 创建/审查/合并 Pull Request
- 管理 Issue 和 Labels
- 查看 Actions 运行状态

依赖：

- git CLI 工具

- GitHub CLI (gh)
- GITHUB_TOKEN 环境变量

weather 技能:

能力:

- 查询全球城市天气
- 天气预报
- 温度/湿度/风速等详细信息

实现方式:

- 通过 `web_search` 或 `web_fetch` 查询天气 API

summarize 技能:

能力:

- 长文档摘要
- 网页内容提取与总结
- 多文档对比摘要

使用场景:

- 用户提供长篇文章需要总结
- 需要从多个来源提取关键信息

skill-creator 技能:

能力:

- 引导用户创建新的自定义技能
- 生成 `SKILL.md` 模板
- 检查技能目录结构

这是一个"元技能"——用来创建其他技能的技能。

8.7 内置工具完整列表

8.7.1 工具总览表

类别	工具名	功能	关键特性
文件	<code>read_file</code>	读取文件内容	支持二进制文件 base64、支持行范围
文件	<code>write_file</code>	创建/覆盖文件	自动创建父目录
文件	<code>edit_file</code>	编辑文件局部内容	基于搜索替换, 比 <code>write_file</code> 精确
文件	<code>list_dir</code>	列出目录内容	支持递归、支持过滤
Shell	<code>exec</code>	执行 Shell 命令	<code>asyncio</code> 异步执行、危险命令拒绝
Web	<code>web_search</code>	网络搜索	支持 Brave/DDG/Tavily 等引擎
Web	<code>web_fetch</code>	获取网页内容	HTML 转纯文本
通信	<code>message</code>	发送消息给用户	唯一可携带 <code>media</code> 的工具
调度	<code>cron</code>	定时任务管理	<code>add/list/remove</code> 三种操作
并发	<code>spawn</code>	启动后台子代理	<code>SubagentManager</code> 管理

8.7.2 文件操作工具

`read_file` —— 读取文件

```
{
  "name": "read_file",
  "parameters": {
    "path": "string (必需) - 文件路径",
```

```

    "start_line": "int (可选) - 起始行号",
    "end_line": "int (可选) - 结束行号"
}
}

```

使用示例:

Agent: [调用 read_file]

参数: {"path": "src/main.py"}

返回: "import os\nimport sys\n\ndef main():\n print('Hello')\n..."

write_file —— 写入文件

```

{
  "name": "write_file",
  "parameters": {
    "path": "string (必需) - 文件路径",
    "content": "string (必需) - 文件内容"
  }
}

```

关键行为: - 如果文件不存在, 自动创建 (包括父目录) - 如果文件已存在, 完全覆盖 - 受 `restrict_to_workspace` 限制

edit_file —— 编辑文件

```

{
  "name": "edit_file",
  "parameters": {
    "path": "string (必需) - 文件路径",
    "old_text": "string (必需) - 要替换的原文本",
    "new_text": "string (必需) - 替换后的新文本"
  }
}

```

关键行为: - 基于精确字符串匹配定位修改位置 - 比 `write_file` 更安全 (不会意外覆盖整个文件) - 如果 `old_text` 找不到匹配, 返回错误

list_dir —— 列出目录

```

{
  "name": "list_dir",
  "parameters": {
    "path": "string (必需) - 目录路径",
    "recursive": "bool (可选) - 是否递归",
    "pattern": "string (可选) - 过滤模式"
  }
}

```

8.7.3 Shell 执行工具

exec —— 执行命令

```

{
  "name": "exec",
  "parameters": {
    "command": "string (必需) - Shell 命令"
  }
}

```

核心实现特性:

简化的 `exec` 工具实现

```
async def exec_tool(command: str, workspace: str) -> str:
```



```

# 1. 危险命令检测
if is_dangerous_command(command):
    return "Error: This command is potentially dangerous and has been blocked."

# 2. 使用 asyncio 异步执行
process = await asyncio.create_subprocess_shell(
    command,
    stdout=asyncio.subprocess.PIPE,
    stderr=asyncio.subprocess.PIPE,
    cwd=workspace # 在 workspace 目录下执行
)

stdout, stderr = await process.communicate()

# 3. 返回结果
return f"Exit code: {process.returncode}\n{stdout.decode()}\n{stderr.decode()}"

```

危险命令拒绝机制:

```

DANGEROUS_PATTERNS = [
    r"rm\s+--rf\s+/",          # 删除根目录
    r"mkfs\.",                # 格式化磁盘
    r"dd\s+if=",              # 磁盘写入
    r">\s*/dev/sd",           # 写入磁盘设备
    r"chmod\s+--R\s+777\s+/", # 全局权限修改
]

def is_dangerous_command(command: str) -> bool:
    for pattern in DANGEROUS_PATTERNS:
        if re.search(pattern, command):
            return True
    return False

```

SSRF 防护:

exec 工具中如果涉及网络请求, 会检查目标地址是否为内网:

```

BLOCKED_HOSTS = [
    "127.0.0.1", "localhost",
    "169.254.169.254", # AWS 元数据服务
    "10.0.0.0/8",      # 内网 A 类
    "172.16.0.0/12",   # 内网 B 类
    "192.168.0.0/16",  # 内网 C 类
]

```

8.7.4 Web 工具

web_search ——网络搜索

```

{
  "name": "web_search",
  "parameters": {
    "query": "string (必需) - 搜索关键词",
    "num_results": "int (可选) - 返回结果数量"
  }
}

```

支持多个搜索引擎:

引擎	说明	需要 API Key
Brave Search	隐私搜索引擎	是
DuckDuckGo	免费搜索	否
Tavily	AI 优化的搜索	是
SearXNG	自托管搜索聚合	否

配置方式：

```
{
  "tools": {
    "web_search": {
      "provider": "brave",
      "api_key": "BSAxxxxxxx"
    }
  }
}
```

web_fetch —— 获取网页

```
{
  "name": "web_fetch",
  "parameters": {
    "url": "string (必需) - 网页 URL"
  }
}
```

核心功能：将 HTML 页面转换为纯文本，方便 Agent 阅读。

输入："https://example.com/article"

处理流程：

1. HTTP GET 请求获取 HTML
2. 解析 HTML，提取正文
3. 移除脚本、样式、导航等无关元素
4. 转换为清晰的纯文本格式
5. SSRF 防护：拒绝内网地址

输出："文章标题\n\n文章正文内容..."

8.7.5 通信工具

message —— 发送消息

```
{
  "name": "message",
  "parameters": {
    "content": "string (必需) - 消息内容",
    "media": "array (可选) - 媒体附件"
  }
}
```

关键特性：- 这是 Agent 主动向用户发送消息的唯一方式 - 对应 `OutboundMessage` 数据结构 - 唯一可以携带 **media** (图片/文件) 的工具

`OutboundMessage` 数据结构

@dataclass

class `OutboundMessage`:

 channel: str # 目标通道

 chat_id: str # 目标会话

```

content: str      # 文本内容
reply_to: str     # 回复的消息 ID (可选)
metadata: dict    # 元数据
media: list       # 媒体附件列表 (图片、文件等)

```

8.7.6 调度工具

cron —— 定时任务

```

{
  "name": "cron",
  "parameters": {
    "action": "string (必需) - add/list/remove",
    "name": "string (action=add/remove 时必需) - 任务名称",
    "schedule": "object (action=add 时必需) - 调度配置",
    "message": "string (action=add 时必需) - 触发时发送的消息"
  }
}

```

详见 10 - 子 Agent 与定时任务。

8.7.7 并发工具

spawn —— 启动子代理

```

{
  "name": "spawn",
  "parameters": {
    "task": "string (必需) - 子代理的任务描述"
  }
}

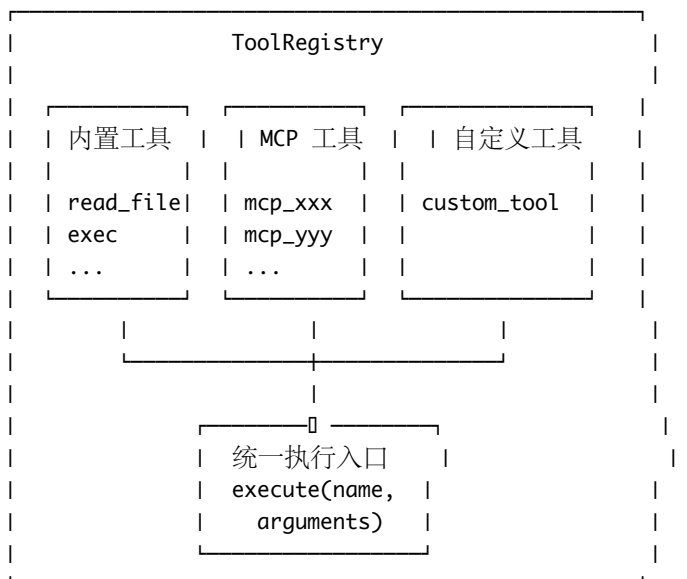
```

详见 10 - 子 Agent 与定时任务。

8.8 ToolRegistry 统一注册与执行机制

8.8.1 ToolRegistry 架构

ToolRegistry 是 Nanobot 工具系统的核心枢纽，负责统一管理所有工具的注册、发现和执行：



8.8.2 注册流程

简化的 ToolRegistry 实现

```
class ToolRegistry:
    def __init__(self):
        self._tools: dict[str, ToolDefinition] = {}

    def register(self, name: str, definition: dict, handler: Callable):
        """ 注册一个工具 """
        self._tools[name] = ToolDefinition(
            name=name,
            definition=definition, # JSON Schema 格式的工具定义
            handler=handler        # 实际执行函数
        )

    def get_definitions(self) -> list:
        """ 获取所有工具定义（用于发送给 LLM） """
        return [
            {
                "type": "function",
                "function": tool.definition
            }
            for tool in self._tools.values()
        ]

    async def execute(self, name: str, arguments: dict) -> str:
        """ 执行一个工具调用 """
        if name not in self._tools:
            return f"Error: Unknown tool '{name}'"

        tool = self._tools[name]
        try:
            result = await tool.handler(**arguments)
            return str(result)
        except Exception as e:
            return f"Error: {str(e)}"
```

8.8.3 执行流程

LLM 返回工具调用请求

|
□

```
| 解析 tool_calls |
| name: "read_file" |
| arguments: {"path": |
|   "src/main.py"} |
```

|
□

```
| ToolRegistry.execute |
| 1. 查找注册的 handler |
| 2. 验证参数 |
| 3. 执行 handler |
| 4. 返回结果 |
```

```

|
|
| 将结果作为 tool
| message 返回给 LLM
| role: "tool"
| content: "文件内容..."
|
|
|
LLM 基于工具结果继续推理

```

8.8.4 工具定义的 JSON Schema 格式

所有工具都遵循 OpenAI 的 Function Calling 格式：

```

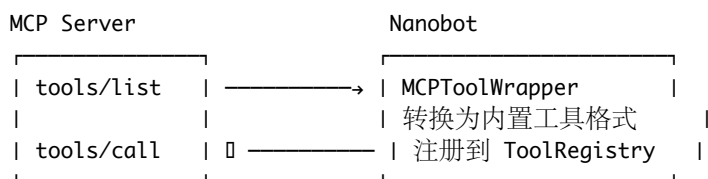
{
  "type": "function",
  "function": {
    "name": "read_file",
    "description": "Read the contents of a file",
    "parameters": {
      "type": "object",
      "properties": {
        "path": {
          "type": "string",
          "description": "The path to the file to read"
        },
        "start_line": {
          "type": "integer",
          "description": "Optional start line number"
        },
        "end_line": {
          "type": "integer",
          "description": "Optional end line number"
        }
      },
      "required": ["path"]
    }
  }
}

```

8.9 MCP 工具集成

8.9.1 MCPToolWrapper

Nanobot 通过 MCPToolWrapper 将 MCP Server 提供的工具“包装”为内置工具格式，实现无缝集成：



8.9.2 转换过程

简化的 MCPToolWrapper 实现

```
class MCPToolWrapper:
    def __init__(self, mcp_client):
        self.client = mcp_client

    async def wrap_tools(self, registry: ToolRegistry):
        """ 将 MCP 工具转换并注册到 ToolRegistry"""
        # 1. 获取 MCP 工具列表
        mcp_tools = await self.client.list_tools()

        for mcp_tool in mcp_tools:
            # 2. 转换工具定义格式
            definition = {
                "name": f"mcp_{mcp_tool.name}",
                "description": mcp_tool.description,
                "parameters": mcp_tool.input_schema
            }

            # 3. 创建执行代理函数
            async def handler(**kwargs):
                result = await self.client.call_tool(
                    mcp_tool.name, kwargs
                )
                return result.content

            # 4. 注册到 ToolRegistry
            registry.register(
                name=f"mcp_{mcp_tool.name}",
                definition=definition,
                handler=handler
            )
```

8.9.3 MCP 工具的命名规范

MCP 工具在注册时会添加前缀以避免命名冲突:

MCP Server 原始工具名: `get_weather`
注册到 ToolRegistry: `mcp_get_weather`

MCP Server 原始工具名: `search_docs`
注册到 ToolRegistry: `mcp_search_docs`

8.9.4 配置 MCP Server

在 config.json 中配置 MCP Server:

```
{
  "mcp": {
    "servers": {
      "weather": {
        "command": "npx",
        "args": ["-y", "@weather/mcp-server"],
        "env": {
          "API_KEY": "xxx"
        }
      },
    },
    "database": {
```

```

        "command": "python",
        "args": ["-m", "db_mcp_server"],
        "env": {
            "DB_URL": "postgresql://localhost/mydb"
        }
    }
}
}
}
}

```

8.10 自定义 Skill 编写实战

8.10.1 实战：创建一个“代码分析”技能

Step 1: 创建技能目录

```
mkdir -p skills/code-analyzer
```

Step 2: 编写 SKILL.md

```

---
name: code-analyzer
description: "Python 代码质量分析与优化建议"
always: false
metadata: '{"nanobot.requires.bins': ['python3']}'
---

```

代码分析师技能

你是一名资深的 Python 代码分析师。当用户请求代码分析时，按以下流程操作。

分析维度

1. ** 代码规范 **: PEP 8 合规性、命名规范
2. ** 安全性 **: SQL 注入、路径遍历、硬编码密钥
3. ** 性能 **: 算法复杂度、不必要的计算、N+1 查询
4. ** 可维护性 **: 函数长度、圈复杂度、耦合度
5. ** 测试覆盖 **: 关键路径是否有测试

分析流程

Step 1: 收集代码

使用 `list_dir` 查看项目结构 使用 `read_file` 读取关键文件

Step 2: 运行静态分析

使用 `exec` 运行以下命令（如果工具可用）： - `python3 -m py_compile` # 语法检查 - `python3 -m pylint` # 代码质量 - `python3 -m bandit` # 安全扫描

Step 3: 生成报告

按以下格式生成分析报告：

报告模板

```
### □ 总体评分：X/10
```

□ 严重问题
(列出所有严重问题)

□ 改进建议
(列出所有改进建议)

□ 优秀实践
(列出代码中做得好的地方)

□ 重构建议
(给出具体的重构方案)

Step 3: 添加参考资料 (可选)

```
mkdir -p skills/code-analyzer/references
```

```
<!-- skills/code-analyzer/references/security-checklist.md -->  
# Python 安全检查清单
```

常见安全漏洞

1. SQL 注入: 使用参数化查询而非字符串拼接
2. 路径遍历: 验证用户输入的文件路径
3. 命令注入: 避免 `os.system()`, 使用 `subprocess`
4. 硬编码密钥: 使用环境变量或密钥管理服务
5. 不安全的反序列化: 避免 `pickle.loads()` 处理不信任数据

Step 4: 添加辅助脚本 (可选)

```
mkdir -p skills/code-analyzer/scripts
```

```
#!/usr/bin/env python3  
# skills/code-analyzer/scripts/complexity.py  
""" 计算 Python 文件的圈复杂度 """  
import ast  
import sys  
  
def calculate_complexity(filepath):  
    with open(filepath) as f:  
        tree = ast.parse(f.read())  
  
    for node in ast.walk(tree):  
        if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):  
            complexity = 1  
            for child in ast.walk(node):  
                if isinstance(child, (ast.If, ast.While, ast.For,  
                                     ast.ExceptHandler, ast.With,  
                                     ast.BoolOp)):  
                    complexity += 1  
            print(f"{node.name}: complexity = {complexity}")  
  
if __name__ == "__main__":  
    calculate_complexity(sys.argv[1])
```

Step 5: 测试技能

```
# 启动 Agent  
nanobot
```



```
# 测试对话:
# You: 请分析一下 src/main.py 的代码质量
# Agent 应该会读取 SKILL.md, 然后按照分析流程执行
```

8.10.2 实战: 创建一个“面试教练”技能

```
---
name: interview-coach
description: "AI Agent 方向的面试训练和评估"
always: false
---
```

面试教练技能

模式

模式 1: 知识测验

随机从知识库中抽取问题, 评估用户的回答。

模式 2: 模拟面试

模拟真实面试场景, 包括追问和压力测试。

模式 3: 答案优化

帮助用户优化已有的面试回答。

评分标准

- 技术准确性 (40%)
- 表达清晰度 (20%)
- 深度和广度 (20%)
- 实际经验体现 (20%)

知识库主题

1. AI Agent 基础概念
2. 框架架构设计
3. 记忆系统
4. 工具与技能系统
5. 多平台部署
6. 安全与生产环境

反馈格式

评分: X/10

**** 优点 **:**

- ...

**** 不足 **:**

- ...

**** 改进建议 **:**

- ...

**** 参考答案 **:**

(给出一个更完善的回答示例)

8.11 工具安全机制

8.11.1 restrict_to_workspace

所有文件操作工具（read_file, write_file, edit_file, list_dir）都受 workspace 限制：

```
def validate_path(path: str, workspace: str) -> str:
    """ 确保路径在 workspace 范围内 """
    abs_path = os.path.abspath(os.path.join(workspace, path))
    abs_workspace = os.path.abspath(workspace)

    if not abs_path.startswith(abs_workspace):
        raise PermissionError(
            f"Access denied: {path} is outside workspace"
        )

    return abs_path
```

8.11.2 exec 工具的安全层

用户命令 → 危险模式检测 → SSRF 检测 → 路径限制 → 执行

	拒绝危险命令	拒绝内网访问	限制 cwd	异步执行
	(rm -rf /)	(127.0.0.1)	(workspace)	
└─	如果 exec.allowed = false, 直接拒绝所有命令			

8.11.3 web_fetch 的 SSRF 防护

```
def is_ssrf_target(url: str) -> bool:
    """ 检查 URL 是否指向内网地址 """
    from urllib.parse import urlparse
    import ipaddress

    parsed = urlparse(url)
    hostname = parsed.hostname

    try:
        ip = ipaddress.ip_address(hostname)
        return ip.is_private or ip.is_loopback or ip.is_reserved
    except ValueError:
        return hostname in ("localhost", "metadata.google.internal")
```

8.12 面试高频题

题目 1: Nanobot 的工具系统是如何设计的？

参考回答：

“Nanobot 的工具系统基于 **ToolRegistry** 统一注册机制。所有工具——无论是内置工具（read_file、exec 等）还是 MCP 外部工具——都通过 ToolRegistry 统一注册和执行。

每个工具包含三部分：工具定义（JSON Schema 格式，描述参数）、执行函数（实际的业务逻辑）、安全约束（权限检查、路径限制等）。

工具定义会作为 Tool Definition 发送给 LLM，LLM 决定何时调用什么工具。调用请求返回后，ToolRegistry 根据工具名找到对应的 handler 执行，并将结果以 tool message 返回给 LLM。

对于 MCP 外部工具，通过 MCPToolWrapper 将 MCP 协议的工具格式转换为内置格式，然后统一注册到 ToolRegistry。这样 Agent 无需区分工具来源，使用方式完全一致。”

题目 2: Skill 和 Tool 的区别是什么?

参考回答:

“Skill 和 Tool 在 Nanobot 中是两个不同层次的概念。

Tool 是细粒度的原子操作，比如 read_file 读文件、exec 执行命令、web_search 搜索网页。每个 Tool 有明确的参数定义和执行逻辑，通过 ToolRegistry 注册，以 JSON Schema 格式暴露给 LLM。

Skill 是粗粒度的能力模块，本质上是一个 Markdown 文件 (SKILL.md)，告诉 Agent ‘你能做什么、怎么做’。一个 Skill 通常会教 Agent 如何组合使用多个 Tool 来完成复杂任务。比如 GitHub Skill 教 Agent 如何组合使用 exec (运行 git 命令) 和 read_file (读取代码) 来完成代码审查。

简单类比: Tool 是锤子、螺丝刀这些工具，Skill 是 ‘如何组装家具’ 的说明书。”

题目 3: 什么是渐进披露? 在 Nanobot 中如何应用?

参考回答:

“渐进披露 (Progressive Disclosure) 是 UI/UX 设计中的经典原则——先展示概要，让用户按需深入细节。Nanobot 将这个原则创造性地应用到了 Agent 的 Prompt 管理中。

具体实现是三层结构:

Tier 1: 所有技能的 name 和 description 组成一个摘要列表，始终注入 System Prompt，大约消耗 100-500 tokens。Agent 通过摘要知道自己有哪些能力。

Tier 2: 当 Agent 判断某个技能与当前任务相关时，主动调用 read_file 读取完整的 SKILL.md，获取详细的使用说明。

Tier 3: 如果需要更深入的信息 (参考文档、辅助脚本)，Agent 继续访问 references/ 和 scripts/ 目录。

唯一的例外是标记了 `always: true` 的技能，它们跳过渐进披露，全文注入 System Prompt。

这种设计的价值在于 token 管理——如果有 20 个技能全部注入，可能消耗 1 万 token；使用渐进披露后，常态只消耗 300 token，按需加载时才产生额外消耗。”

题目 4: 如何为 Nanobot 添加一个新的工具?

参考回答:

“有三种方式:

第一种是写 **Skill**——不需要写代码，只需创建一个 SKILL.md 文件，用 Markdown 描述这个能力的使用方式。Agent 会基于已有的 Tool (exec、web_fetch 等) 来执行。这适合流程性、指导性的扩展。

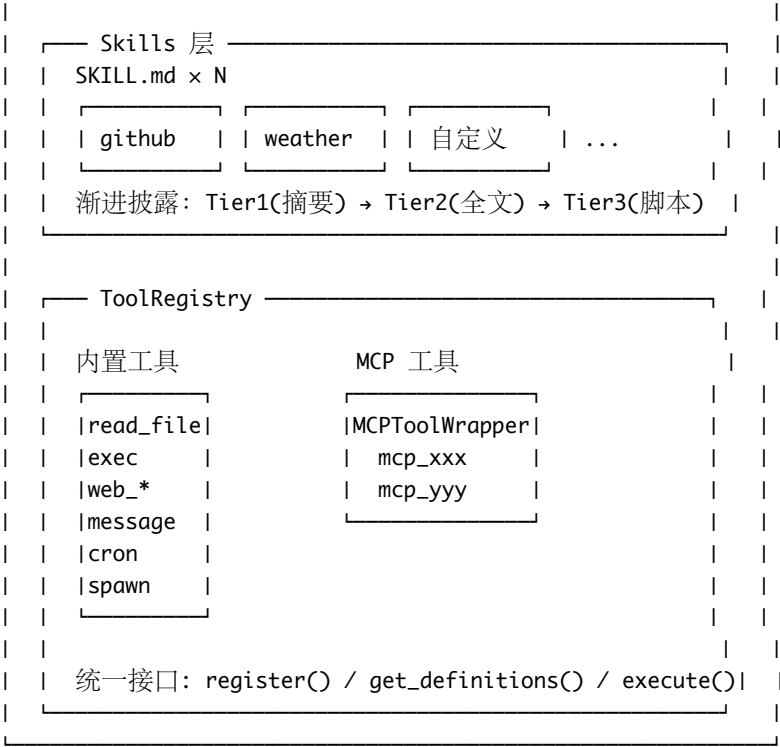
第二种是接入 **MCP Server**——如果需要专用的 API 调用或复杂逻辑，可以开发一个 MCP Server，Nanobot 通过 MCPToolWrapper 自动将其工具转换为内置格式注册到 ToolRegistry。

第三种是修改源码——在 ToolRegistry 中直接注册新的工具函数。这种方式最灵活但需要修改框架代码，不太适合分发。

推荐优先级: Skill > MCP Server > 源码修改。Skill 是最轻量方式，MCP Server 提供了标准化的扩展接口。”

8.13 本章小结

核心架构图



面试记忆清单

考点	一句话回答
Skill vs Tool	Skill 是能力说明书 (Markdown), Tool 是具体操作 (代码)
SKILL.md 格式	YAML Frontmatter + Markdown 正文
渐进披露	三层: 摘要目录 → 全文读取 → 脚本/参考
技能发现	workspace/skills/ 优先, 同名覆盖内置
ToolRegistry	统一注册与执行所有工具 (内置 + MCP)
MCPToolWrapper	将 MCP 工具转换为内置格式
安全机制	restrict_to_workspace + 危险命令拒绝 + SSRF 防护
always 字段	true= 全文注入, false= 仅摘要 (默认)

下一章: 09 - 多平台接入 —— 了解 Nanobot 如何同时接入 Telegram、Discord、飞书、钉钉等 8+ 平台